void SymbolTable::ScopeTable::addIdentifier(Identifier* ident) -> use getName() method to get string name. Then insert {name, ident} pair into symbolTable hash map

Identifier* SymbolTable::ScopeTable::searchInScope(const char* name) -> search symbolTable hashmap for element with key=name. Cannot use mSymbols[name] because if no such element exists then a new entry with value=nullptr is created in the table. Alternative is to use mSymbols.find(name) and return the element found

Identifier* SymbolTable::ScopeTable::search(const char* name) -> make local ScopeTable* localTable = this . Iterate through localTable while the pointer is not null since parent of root pointer is null. Call localTable->searchInScope(name) in each iteration and assign value to retVal. If retVal is not null at any iteration, return it. Else move to the parent of the local table via localTable = localTable->getParent(). If no retVal is found return nullptr

SymbolTable::ScopeTable::ScopeTable(ScopeTable* parent) noexcept : mParent(parent) -> Parent->mChild is a list, so we must push the current instance of the object to the back of the list

SymbolTable::ScopeTable* SymbolTable::enterScope() -> this is contingent on ScopeTable constructor, because we need to create a new Scopetable and point mCurrScope at it so that the symbol table knows the current scope. Then we can return mCurrScope

SymbolTable::ScopeTable::~ScopeTable() noexcept -> iterate through mSymbols and delete every value as each value was dynamically allocated. Then iterate through mChildren and delete every element as they were also dynamically allocated. mParent needn't be cleaned up as the parent owns the children and it will be automatically discarded

void SymbolTable::exitScope() -> find the parent of the current scope. Simple

SymbolTable::~SymbolTable() noexcept -> check if mCurrTable is not nullptr. If it isn't then create a local table with the same scope and call getParent() until the root. then delete the local table

SymbolTable::SymbolTable() noexcept : mCurrScope(nullptr) -> enter current scope and then create three new identifiers ('@@function', '@@variable', 'printf'). Then add identifiers to current scope.

Identifier* SymbolTable::createIdentifier(const char* name) -> add identifier to mCurrScope using addIdentifier()

bool SymbolTable::isDeclaredInScope(const char* name) const noexcept -> check if current scope is not null since we only need to check the current and not parent scopes. If true then check if searchInScope returns a non null value. If both conditions satisfied, return true else return false

Identifier* SymbolTable::getIdentifier(const char* name) -> check if mCurrScope is nullptr. If yes then return nullptr. otherwise search the current scope up to parents. If search returns a non null value, then return that. Otherwise return nullptr

Identifier* Parser::getVariable(const char* name) noexcept -> we need to first look up the symbol table using the getIdentifier(const char* name) method. If it returns a non null value, the variable exists so we return it. If it returns a nullptr, then we build the "Use of undeclared identifier 'x'" error message where 'x' is the name and then report it using reportSemantError(msg). We then return the 'dummy' @@variable value by using getIdentifier("@@variable")

shared_ptr<ASTCompoundStmt> Parser::parseCompoundStmt(bool isFuncBody) -> logically we must only enter the current scope if the scope is declared, meaning after a '{' sign. Similarly, we must exit the scope before the '}' in case an exception is thrown. The rest of the function can remain unchanged

shared_ptr<ASTDecl> Parser::parseDecl() -> inside of the try-catch block, use check if the current token text is declared within current scope using isDeclaredInScope. If it is, then build the error message and report semantic error. Otherwise create the identifier

std::shared_ptr<ASTExpr> Parser::charToInt(std::shared_ptr<ASTExpr> expr) noexcept -> if expr is null, return the nullptr. first we need to check if the type is char. If it isn't then return retVal as it is. if it is a char type then we need to check via std::dynamic_pointer_cast<ASTConstantExpr>(expr) to see if it is a char constant or another char type. If it is a char constant then can call the changeToInt() function. If not then convert to make_shared<ASTConstantExpr>(expr) and return the value

std::shared_ptr<ASTExpr> Parser::intToChar(std::shared_ptr<ASTExpr> expr) noexcept -> same logic as charToInt() but in reverse. intToChar first checks whether the expression's type is Int. If it isn't, it returns the expression unchanged. If it is, it uses std::dynamic_pointer_cast<ASTToIntExpr>(expr) to check whether the expression is a conversion wrapper that was created by charToInt. If it is, the child inside that wrapper

is already a char, so it returns getChild() instead of adding another conversion. Otherwise it wraps the expression in an ASTToCharExpr and returns that

shared_ptr<ASTExpr> Parser::parseIncFactor() -> to integrate charToInt() simply call charToInt(retVal) and assign it to retVal before the return statement.

shared_ptr<ASTExpr> Parser::parseDecFactor() -> exact same logic as parseIncFactor()

shared_ptr<ASTStmt> Parser::parseAssignStmt(bool isForStep) -> handles assignments to plain variables and to array elements, and it does several semantic checks along the way. After the identifier is read, if a '[' follows and the identifier isn't an array (and isn't the @@variable dummy), it reports "x is not an array", using a column saved before the '[' was consumed so it lands on the bracket. It then skips to the closing ']' and consumes it, so the rest of the statement parses as a plain assignment and only one error is reported.
After the right-hand expression is parsed, the function compares types. For an array element, the expected type comes from the array's element type. For a plain variable, it comes from ident->getType(). If the variable is a char and the expression is an int, intToChar converts it. Any other mismatch reports "Cannot assign an expression of type X to Y" at the column of the '='. If the target itself is an array (a = b;), it instead reports "Reassignment of arrays is not allowed" and skips the type check, so an array target produces exactly one error. The checks all run before the assignment node is built, so the node stores the converted expression

shared_ptr<ASTDecl> Parser::finishDeclarator(Identifier& ident, Type baseType) -> After an ordinary initializer expr, not a list is parsed, the declared type is compared with the expression's type. A char initialized from an int is converted with intToChar, and any other mismatch reports "Cannot assign an expression of type X to Y" at the column of the '='. A char array initialized from a string has matching types, so it passes, and the existing string-size checks then run

shared_ptr<ASTReturnStmt> Parser::parseReturnStmt() -> For "return;", a non-void function reports Invalid empty return in non-void function. For "return expr;", the expression's type is compared with the function's return type (mCurrReturnType). An int returned from a char function is converted with intToChar, and any other mismatch reports "Expected type X in return statement", where X is the function's return type. The check only runs when the expression parsed successfully, and it happens before the ASTReturnStmt node is built

bool Parser::checkInitializerList(Identifier& ident, shared_ptr<ASTArrayInitExpr> init, size_t dimIdx) -> First it rejects two structural errors. If the identifier isn't an array, it reports “Initializer list is not allowed for non-array ‘a’”. If dimIdxis already at or past the number of dimensions, the list is nested deeper than the array, so it reports “Nested initializer is too deep for array ‘b’”. Then it counts how many elements are nested lists (found with dynamic_pointer_cast<ASTArrayInitExpr>) and how many are plain expressions. An empty list is valid. A list containing both kinds reports “Cannot mix braced and unbraced initializers for array ‘e’”.
If every element is a list, the number of lists can't exceed the extent of the current dimension (Too many initializers), and each nested list is checked by a recursive call with dimIdx + 1. If every element is a plain expression, the braces below this level were left out, so the count can't exceed the product of the extents from dimIdx to the last dimension, and each element's type is checked against the array's element type. An Int going into a Char is converted with intToChar, and anything else reports “Cannot assign an expression of type X to Y”. The function returns false as soon as one error is reported, so each declaration produces only one error

ASTLogicalAnd::finalizeOp(), ASTLogicalOr::finalizeOp(), ASTBinaryCmpOp::finalizeOp() and ASTBinaryMathOp::finalizeOp() -> all four functions have same exact structure. Set current type to int and return true iff both LHS and RHS are integer type themselves

shared_ptr<ASTLogicalOr> Parser::parseExprPrime(shared_ptr<ASTExpr> lhs), shared_ptr<ASTBinaryCmpOp> Parser::parseRelExprPrime(shared_ptr<ASTExpr> lhs), shared_ptr<ASTBinaryMathOp> Parser::parseNumExprPrime(shared_ptr<ASTExpr> lhs) and shared_ptr<ASTBinaryMathOp> Parser::parseTermPrime(shared_ptr<ASTExpr> lhs) -> Each function builds a binary operator node once it sees its operator token, and each does the semantic check the same way. It records the column of the operator before consuming it, sets the left operand, parses the right operand, and sets the right operand. Then it calls finalizeOp() on the new node, which sets the node's type to Int and returns whether both operands are Int. finalizeOp() is always called, so the node gets its type even if the operands are bad. If it returns false, the function reports “Cannot perform op between type X and Y” (the left and right operand types), using the saved column so it lands on the operator. The check happens after both operands are set and before the recursive call. That way, in a chain like a + b + c, the inner node already has type Int when it becomes the left operand of the next link

shared_ptr<ASTCompoundStmt> Parser::parseCompoundStmt(bool isFuncBody) -> After the opening ’{‘ is consumed, the function enters a new symbol table scope, unless isFuncBody is true. In that case parseFunction has already created the scope, so the

function's parameters and its body share one scope. Declarations and statements are then parsed, and the last statement is kept in lastStmt.
For a function body, the last statement is tested with std::dynamic_pointer_cast<ASTReturnStmt>. If it isn't a return, or if the body has no statements, the outcome depends on mCurrReturnType. A non-void function reports “USC requires non-void functions to end with a return”. A void function gets an empty ASTReturnStmt appended, so later phases always find a terminating return. The scope is then exited for a non-function block. This happens before the closing ‘}’ is matched, so that if matchToken throws, the symbol table is still balanced

bool SymbolTable::checkShadow(const char* name) -> It returns true if declaring name in the current scope would hide a declaration in an enclosing scope. Starting at the current scope's parent, it walks upward through every enclosing scope, including the root (global) scope. In each scope it looks the name up with searchInScope, and a non-null result whose type isn't Function counts as a shadowed variable, so the function returns true. A match that is a function is skipped, because a variable that reuses a function's name doesn't shadow it, and the walk continues so that a real variable further out is still found. If it reaches the top without a match, it returns false. The current scope isn't searched, since a repeat there is the redeclaration error and not shadowing

shared_ptr<ASTDecl> Parser::parseDecl() -> declaration must start with int or char, so the function first reads the base type and consumes that keyword. It sets ident to the @@variable dummy as a fallback, so the parse can continue if the declaration is wrong. Inside a try, it requires an identifier token and reads its name without consuming it yet. If the name is already declared in the current scope, it reports “Invalid redeclaration of identifier ‘x’” and keeps the dummy, so no second identifier is created. Otherwise, it calls checkShadow, and if the name hides a variable from an enclosing scope, it reports “warning: declaration of 'x' shadows a previous declaration” using reportSemantWarn, so it goes in the warning list and doesn't count as an error. Only then does it create the identifier in the current scope. Both messages are reported before the identifier token is consumed, so it lands on the name. It then consumes the identifier, calls finishDeclarator to handle array bounds and any initializer, and matches the ‘;’. If a syntax error is thrown, the catch reports it, skips to the next terminator or statement start, and returns a declaration node built on the fallback identifier, so parsing can continue with the next declaration.

shared_ptr<ASTExprStmt> Parser::parseExprStmt() -> After the expression is parsed, the function checks whether it's a function call by trying dynamic_pointer_cast<ASTFuncExpr>. If it is, it takes the function's name from the call node and looks it up in the symbol table, then reads the function node from that

identifier with getFunction(). If a function node exists and its return type isn't void, the call's value is being thrown away, so it reports “warning: ignoring return value of function 'name'” using reportSemantWarn. The lookup is what keeps printf and calls to undeclared names. Printf has no function node. The warning is reported before the ‘;’ is matched, so we land on the semicolon. The function then builds the statement node and matches the ‘;’ as before.

bool Parser::foldConstant(shared_ptr<ASTExpr> expr, int& out) ->
It tries to evaluate an expression to a single integer at compile time, returning true and storing the value in ‘out’ if it can, and false if it can't. A constant node yields its own value. A binary math node recursively folds its left and right children, and if both succeed, combines the two values using the node's math operator. Division and modulo by zero are caught by an if statement, so the function returns false instead of computing. Anything else, such as a variable or a function call, has no compile-time value, so it returns false. out is only written when the fold succeeds.

bool SymbolTable::checkArrayIdx(const char* name, std::shared_ptr<ASTExpr> expr, size_t dimIdx) ->
It reports whether a constant index is outside the bounds of an array's dimension, returning true if it is. It first checks that the expression is an ASTConstantExpr, since a non-constant index can't be checked. It then looks up the array by name, and returns false if the name isn't found or isn't an array, so a non-array doesn't get a second, unrelated warning. Finally, it compares the index value with the extent of dimension dimIdx, obtained from getExtent. The index is out of range if it's negative or greater than or equal to the extent. The negative check comes first, so a negative int is never compared against the unsigned extent.

void Parser::checkSubscript(Identifier& ident, shared_ptr<ASTExpr> expr, size_t dimIdx) ->
It emits the out-of-bounds warning for one subscript. It first calls foldConstant on the index, and it returns immediately if the index isn't a compile-time constant, such as a[i]. Because checkArrayIdx only accepts a bare constant node, the folded value is wrapped in a fresh ASTConstantExpr used only for the check, so the original expression in the AST is left unchanged. If checkArrayIdx reports that the index is out of range, it displays “warning: index N is outside bounds of array 'a' of size S” through reportSemantWarn, where the size is the extent of that dimension. Each subscript of a multi-dimensional access is checked against its own dimension, and the caret lands on the closing ‘]’.